
Cómputo Nube y Tecnologías Emergentes

Instituto Nacional de Astrofísica, Óptica y Electrónica (INAOE) - Maestría en Ciencias en Tecnologías de Seguridad
Lázaro Bustio-Martínez, PhD
lbustio@inaoe.mx
Verano 2026
96 horas
- Tutorial -

1. Objetivo de la práctica

El objetivo de esta práctica es construir un flujo completo de Machine Learning en Google Cloud Platform (GCP), desde el almacenamiento de datos hasta el despliegue de un modelo como servicio web.

Al finalizar la práctica, el estudiante será capaz de:

- utilizar Cloud Storage para almacenar datasets;
- utilizar Colab Enterprise como entorno de desarrollo;
- cargar datos desde Cloud Storage;
- preparar datos para Machine Learning;
- entrenar un modelo supervisado;
- evaluar el desempeño del modelo;
- guardar un modelo entrenado;
- almacenar el modelo en Cloud Storage;
- desplegar el modelo mediante Cloud Run;
- consumir el modelo a través de una API REST.

2. Arquitectura de la solución

La práctica sigue una arquitectura mínima, reproducible y orientada al aprendizaje. El dataset se almacena en Cloud Storage, el entrenamiento ocurre en Colab Enterprise y el modelo resultante se despliega como servicio HTTP mediante Cloud Run.

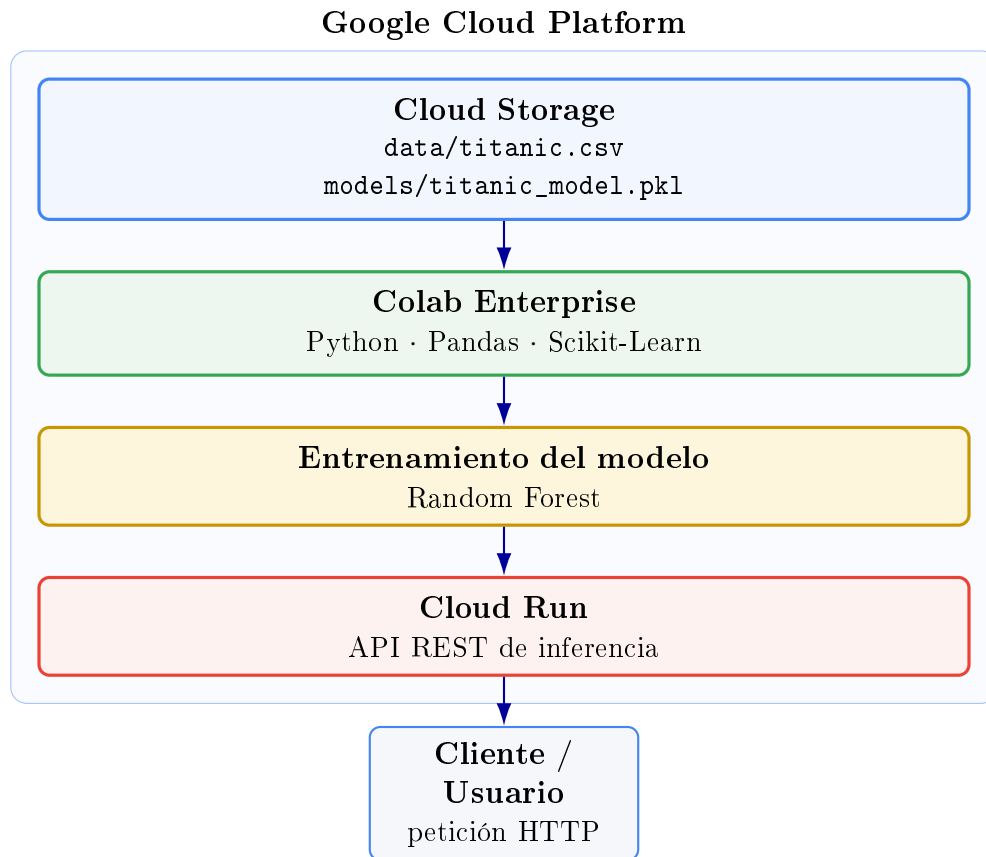


Figura: Arquitectura general del flujo de Machine Learning en GCP.

3. Tecnologías utilizadas

3.1. Google Cloud Platform

Google Cloud Platform (GCP) es la plataforma de computación en la nube desarrollada por Google. Proporciona infraestructura, almacenamiento, inteligencia artificial, redes y servicios administrados.

3.2. Cloud Storage

Cloud Storage es el servicio de almacenamiento de objetos de Google Cloud. En esta práctica se utilizará para almacenar:

- el dataset Titanic;
- el modelo entrenado.

3.3. Colab Enterprise

Colab Enterprise es un entorno administrado basado en notebooks Jupyter. En esta práctica se utilizará para:

- ejecutar código Python;
- analizar datos;
- entrenar modelos;
- evaluar resultados.

3.4. Python

Python será el lenguaje de programación utilizado para implementar todo el flujo de trabajo.

3.5. Pandas

Pandas es una biblioteca especializada en manipulación y análisis de datos. Se utilizará para cargar, explorar y preparar el dataset.

3.6. Scikit-Learn

Scikit-Learn es una biblioteca de Machine Learning que se utilizará para:

- dividir datos;
- entrenar el modelo;
- evaluar resultados;
- generar predicciones.

3.7. Cloud Run

Cloud Run es un servicio serverless utilizado para desplegar aplicaciones contenidas en Docker. En esta práctica se usará para publicar una API de inferencia.

4. Preparación del proyecto

Antes de iniciar el laboratorio, realice las siguientes verificaciones:

1. Acceda a Google Cloud Console.
2. Seleccione el proyecto de trabajo.
3. Verifique que la facturación se encuentre habilitada.
4. Verifique que estén disponibles los siguientes servicios:
 - Cloud Storage;
 - Colab Enterprise;
 - Cloud Run;
 - Artifact Registry;
 - Cloud Build.

5. Creación del bucket

Desde Google Cloud Console, navegue a:

Cloud Storage → Buckets → Crear bucket

Asigne un nombre único al bucket. Para efectos de esta guía se utilizará el siguiente nombre de ejemplo:

```
1 titanic-tutorial
```

Una vez creado el bucket, genere la siguiente estructura lógica:

```
1 data/  
2 models/
```

Suba el dataset en la siguiente ruta:

```
1 data/titanic.csv
```

Resultado esperado: El bucket debe contener el archivo `data/titanic.csv`.

6. Creación del notebook

Desde Google Cloud Console, navegue a:

Plataforma de agentes → Notebooks → Colab Enterprise

Cree un notebook nuevo con el siguiente nombre sugerido:

```
1 Titanic_ML_GCP
```

7. Verificación del entorno

Verifique la versión de Python disponible en el runtime:

```
1 import sys
2
3 print(sys.version)
```

Verifique también que estén disponibles las bibliotecas principales:

```
1 import pandas as pd
2 import sklearn
3
4 print("Pandas:", pd.__version__)
5 print("Scikit-Learn:", sklearn.__version__)
```

8. Dónde se ejecuta el notebook

Aunque el notebook se abre desde el navegador, el código no se ejecuta en la computadora local. El navegador únicamente muestra la interfaz. La ejecución real ocurre dentro de un runtime administrado por Google Cloud.

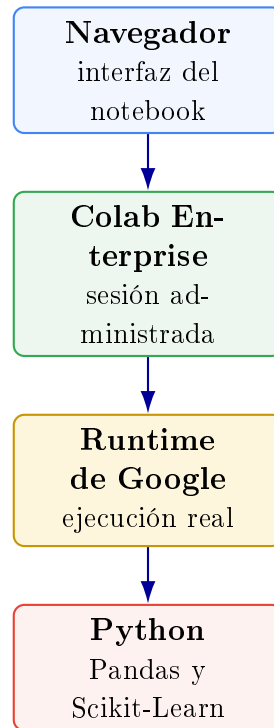


Figura: Separación entre la interfaz web y el entorno real de ejecución.

Verifique el entorno desde una celda del notebook:

```
1 !hostname
2 !nproc
3 !free -h
```

Conclusión operativa

El dataset vive en Cloud Storage, el código vive en el notebook, la ejecución ocurre en Colab Enterprise y los archivos locales del runtime son temporales.

9. Carga del dataset

Importe Pandas:

```
1 import pandas as pd
```

Cargue el archivo desde Cloud Storage:

```
1 df = pd.read_csv(  
2     "gs://titanic-tutorial/data/titanic.csv"  
3 )
```

Muestre los primeros registros:

```
1 df.head()
```

Verifique las dimensiones del dataset:

```
1 df.shape
```

Muestre las columnas disponibles:

```
1 df.columns
```

Obtenga información general sobre los tipos de datos y valores faltantes:

```
1 df.info()
```

10. Comprensión del problema

El objetivo del modelo es predecir la supervivencia de pasajeros del Titanic.

Variable objetivo: Survived

Valor 0: el pasajero no sobrevivió.

Valor 1: el pasajero sobrevivió.

11. Preparación de los datos

Defina la variable objetivo:

```
1 y = df["Survived"]
```

Seleccione las variables predictoras:

```
1 X = df[  
2     [  
3     "Pclass",  
4     "Sex",
```

```
5     "Age",  
6     "SibSp",  
7     "Parch",  
8     "Fare"  
9 ]  
10 ]
```

Revise los valores faltantes:

```
1 X.isnull().sum()
```

Rellene las edades faltantes usando la mediana:

```
1 X["Age"] = X["Age"].fillna(  
2     X["Age"].median()  
3 )
```

Transforme la variable categórica Sex a valores numéricos:

```
1 X["Sex"] = X["Sex"].map(  
2     {  
3         "male": 0,  
4         "female": 1  
5     }  
6 )
```

Verifique el resultado:

```
1 X.head()
```

12. División entrenamiento/prueba

Importe la función de división de datos:

```
1 from sklearn.model_selection import train_test_split
```

Divida los datos en entrenamiento y prueba:

```
1 X_train, X_test, y_train, y_test = train_test_split(  
2     X,  
3     y,  
4     test_size=0.2,
```

```
5     random_state=42
6 )
```

Verifique los tamaños de ambos subconjuntos:

```
1 print(X_train.shape)
2 print(X_test.shape)
```

13. Entrenamiento del modelo

Importe el algoritmo:

```
1 from sklearn.ensemble import RandomForestClassifier
```

Cree el modelo:

```
1 model = RandomForestClassifier(
2     n_estimators=100,
3     random_state=42
4 )
```

Entrene el modelo:

```
1 model.fit(
2     X_train,
3     y_train
4 )
```

Genere predicciones sobre el conjunto de prueba:

```
1 y_pred = model.predict(X_test)
```

Revise las primeras predicciones:

```
1 y_pred[:10]
```

14. Evaluación del modelo

Calcule el accuracy:

```
1 from sklearn.metrics import accuracy_score
```

```
2
3 accuracy = accuracy_score(
4     y_test,
5     y_pred
6 )
7
8 print(f"Accuracy: {accuracy:.4f}")
```

Calcule la matriz de confusión:

```
1 from sklearn.metrics import confusion_matrix
2
3 cm = confusion_matrix(
4     y_test,
5     y_pred
6 )
7
8 print(cm)
```

Visualice la matriz de confusión:

```
1 from sklearn.metrics import ConfusionMatrixDisplay
2 import matplotlib.pyplot as plt
3
4 ConfusionMatrixDisplay(cm).plot()
5
6 plt.show()
```

15. Importancia de variables

Construya una tabla con la importancia de cada variable:

```
1 importance = pd.DataFrame(
2     {
3         "Variable": X.columns,
4         "Importance": model.feature_importances_
5     }
6 )
7
8 importance.sort_values(
9     "Importance",
```

```
10     ascending=False
11 )
```

Preguntas de análisis:

- ¿Qué variable fue la más importante?
- ¿Qué variable fue la menos importante?

16. Predicción sobre un nuevo pasajero

Cree un registro nuevo con los atributos de un pasajero hipotético:

```
1 new_passenger = pd.DataFrame(
2     {
3         "Pclass": [1],
4         "Sex": [1],
5         "Age": [25],
6         "SibSp": [0],
7         "Parch": [0],
8         "Fare": [100]
9     }
10 )
```

Genere la predicción:

```
1 prediction = model.predict(
2     new_passenger
3 )
4
5 print(prediction)
```

Obtenga las probabilidades asociadas a cada clase:

```
1 probability = model.predict_proba(
2     new_passenger
3 )
4
5 print(probability)
```

17. Guardado del modelo

Importe joblib y serialice el modelo entrenado:

```
1 import joblib
2
3 joblib.dump(
4     model,
5     "titanic_model.pkl"
6 )
```

Verifique que el archivo fue creado:

```
1 import os
2
3 os.path.exists(
4     "titanic_model.pkl"
5 )
```

18. Subida del modelo a Cloud Storage

Importe el cliente de Cloud Storage:

```
1 from google.cloud import storage
```

Configure el bucket de trabajo:

```
1 bucket_name = "titanic-tutorial"
2
3 storage_client = storage.Client()
4
5 bucket = storage_client.bucket(
6     bucket_name
7 )
```

Prepare el objeto remoto y suba el modelo:

```
1 blob = bucket.blob(
2     "models/titanic_model.pkl"
3 )
4
5 blob.upload_from_filename(
```

```
6     "titanic_model.pkl"
7 )
8
9 print(
10     "Modelo subido correctamente"
11 )
```

Resultado esperado: El bucket debe contener los siguientes objetos:

```
1 data/titanic.csv
2 models/titanic_model.pkl
```

19. Despliegue del modelo

El objetivo de esta fase es convertir el modelo entrenado en una API REST. La arquitectura lógica del servicio será:

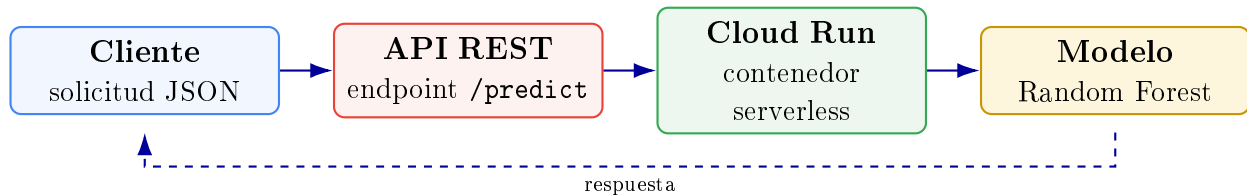


Figura: Ruta lógica de una solicitud de inferencia desplegada en Cloud Run.

20. Creación de app.py

Cree el archivo `app.py` con el siguiente contenido:

```
1 from flask import Flask, request, jsonify
2 import pandas as pd
3 import joblib
4 from google.cloud import storage
5 import os
6
7 app = Flask(__name__)
8
9 BUCKET_NAME = "titanic-tutorial"
10 MODEL_BLOB_PATH = "models/titanic_model.pkl"
11 LOCAL_MODEL_PATH = "/tmp/titanic_model.pkl"
12
13 def download_model():
```

```
14     storage_client = storage.Client()
15     bucket = storage_client.bucket(BUCKET_NAME)
16     blob = bucket.blob(MODEL_BLOB_PATH)
17     blob.download_to_filename(LOCAL_MODEL_PATH)
18
19 download_model()
20
21 model = joblib.load(LOCAL_MODEL_PATH)
22
23 @app.route("/", methods=["GET"])
24 def health_check():
25     return jsonify({"status": "ok"})
26
27 @app.route("/predict", methods=["POST"])
28 def predict():
29
30     data = request.get_json()
31
32     input_data = pd.DataFrame([
33         "Pclass": data["Pclass"],
34         "Sex": data["Sex"],
35         "Age": data["Age"],
36         "SibSp": data["SibSp"],
37         "Parch": data["Parch"],
38         "Fare": data["Fare"]
39     ])
40
41     prediction = model.predict(input_data)[0]
42
43     probability = model.predict_proba(
44         input_data
45     )[0].tolist()
46
47     return jsonify({
48         "prediction": int(prediction),
49         "probability_no_survived": probability[0],
50         "probability_survived": probability[1]
51     })
52
53 if __name__ == "__main__":
54     port = int(
55         os.environ.get("PORT", 8080)
```

```
56     )  
57  
58     app.run(  
59         host="0.0.0.0",  
60         port=port  
61     )
```

21. Creación de requirements.txt

Cree el archivo `requirements.txt` con las dependencias de la API:

```
1 flask  
2 pandas  
3 scikit-learn  
4 joblib  
5 google-cloud-storage  
6 gunicorn
```

22. Creación del Dockerfile

Cree el archivo `Dockerfile` con el siguiente contenido:

```
1 FROM python:3.11-slim  
2  
3 WORKDIR /app  
4  
5 COPY requirements.txt .  
6  
7 RUN pip install \  
8     --no-cache-dir \  
9     -r requirements.txt  
10  
11 COPY app.py .  
12  
13 CMD exec gunicorn \  
14     --bind :$PORT \  
15     --workers 1 \  
16     --threads 8 \  
17     app:app
```

23. Despliegue en Cloud Run

Desde el directorio que contiene `app.py`, `requirements.txt` y `Dockerfile`, ejecute:

```
1 gcloud run deploy titanic-ml-api \  
2   --source . \  
3   --region us-central1 \  
4   --allow-unauthenticated
```

Espere a que finalice el despliegue. Cloud Run devolverá una URL pública para consumir el servicio.

24. Prueba del servicio

Verifique el estado de la API:

```
1 curl https://URL_DEL_SERVICIO
```

Resultado esperado:

```
1 {"status":"ok"}
```

Realice una predicción:

```
1 curl -X POST \  
2 "https://URL_DEL_SERVICIO/predict" \  
3 -H "Content-Type: application/json" \  
4 -d '{  
5     "Pclass":1,  
6     "Sex":1,  
7     "Age":25,  
8     "SibSp":0,  
9     "Parch":0,  
10    "Fare":100  
11  }'
```

Resultado esperado:

```
1 {  
2   "prediction": 1,  
3   "probability_no_survived": 0.02,  
4   "probability_survived": 0.98  
5 }
```

25. Conclusiones

En esta práctica se implementó un flujo completo de Machine Learning en Google Cloud Platform:

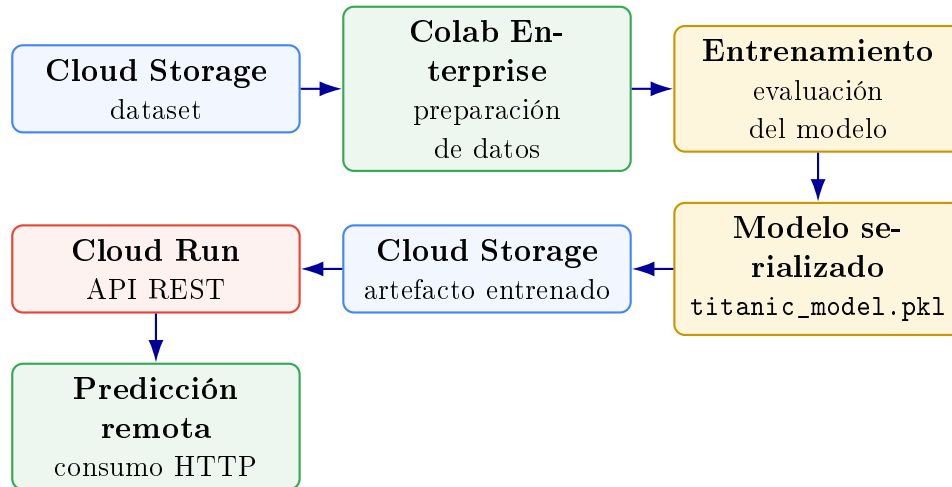


Figura: Pipeline completo de datos, entrenamiento, despliegue y consumo remoto.

Este flujo representa una arquitectura simplificada de producción utilizada en numerosas aplicaciones modernas de Inteligencia Artificial.